

```

    item  := @attrName(attrArg*)def
    def   := importDef | typeDef | constDef | relaDef | queryDef
importDef := import file
typeDef  := type typeName = aliasName | type typeName <: superTypeName
          | type relationName([name:] type*)
constDef := const constName[: type] = constant
relaDef  := rel [tag ::] relaName(expr*) | rel relaName = {taggedTuple*}
          | rel [tag ::] atom :- formula
taggedTuple := [tag ::] (constant*)
atom       := relaName(expr*)
formula    := formula and formula | formula or formula | formula implies formula
          | atom | ¬atom | constraint | reduce
reduce     := var* = aggregator(var*: formula [where var*: formula])
          | var* = sampler(var*: formula [where var*: formula])
constraint := expr
type       := u8 | u16 | u32 | u64 | usize | i8 | i16 | i32 | i64 | isize | f32 | f64 | bool | char | String
expr       := expr binOp expr | unaryOp expr | expr as type | if expr then expr else expr
          | $func(expr*) | variable | constant
binOp      := + | - | * | / | % | && | || | == | != | < | <= | > | >=
unaryOp    := ! | -
aggregator := min | max | argmin<var*> | argmax<var*> | count | sum | prod | exists | forall
sampler    := top<K> | categorical<K> | uniform<K>

```

Fig. 20. Language grammar (var : variable and f : formula).

(Tag)	t	$\in T$
(False)	0	$\in T$
(True)	1	$\in T$
(Disjunction)	$\oplus$	$: T \times T \rightarrow T$
(Conjunction)	$\otimes$	$: T \times T \rightarrow T$
(Negation)	$\ominus$	$: T \rightarrow T$
(Saturation)	$\ominus$	$: T \times T \rightarrow \text{Bool}$
(Early Removal)	discard	$: T \rightarrow \text{Bool}$
(Weight)	weight	$: T \rightarrow \mathbb{R}$

Fig. 21. Full interface for a provenance structure.

A SCALLOP LANGUAGE SYNTAX

In this appendix, we provide the abstract syntax of the full language of Scallop in Fig. 20.

B REASONING FRAMEWORK

In this appendix, we provide the extensions to our provenance framework, and illustrate the additional features provided by full SCLRAM.

B.1 Provenance Extensions

We present the full provenance interface including two additional features, early removal and weight for sampling, in Fig. 21. For early removal, we introduce a function *discard* where it returns a boolean value based on a tag. When returned true, the fact associated with this tag will be early removed from the computation. For sampling, we allow each provenance to implement a function *weight* that returns a real number in $\mathbb{R}$ representing the weight of the tag. These features will be used later when we introduce the full SCLRAM syntax and semantics.

(Predicate)	p	
(Aggregator)	g	$::= \text{count} \mid \text{sum} \mid \text{prod} \mid \text{exists} \mid \text{max} \mid \text{min} \mid \text{argmax} \mid \text{argmin}$
(Sampler)	μ	$::= \text{top}\langle K \rangle \mid \text{categorical}\langle K \rangle \mid \text{uniform}\langle K \rangle$
(Expression)	e	$::= \emptyset \mid p \mid \mathbb{1}(e) \mid \emptyset(e) \mid \gamma_g(e) \mid \hat{\gamma}_g(e_1, e_2) \mid \psi_\mu(e) \mid \hat{\psi}_\mu(e_1, e_2) \mid \pi_\alpha(e) \mid \sigma_\beta(e)$ $\mid e_1 \cup e_2 \mid e_1 \times e_2 \mid e_1 \bowtie e_2 \mid e_1 \cap e_2 \mid e_1 - e_2$
(Rule)	r	$::= p \leftarrow e$
(Stratum)	s	$::= \{r_1, \dots, r_n\}$
(Program)	$\bar{s}$	$::= s_1; \dots; s_n$

Fig. 22. Full abstract syntax of SCLRAM.

B.2 Abstract Syntax of SCLRAM

We revisit the abstract syntax of SCLRAM, and add extensions including:

- (1) Empty-set ($\emptyset$), One-overwrite ($\mathbb{1}(e)$) and zero-overwrite ($\emptyset(e)$),
- (2) Intersection ($\cap$), Natural-Join ($\bowtie$), and Anti-join ($\bar{\bowtie}$),
- (3) Group-by aggregate ($\hat{\gamma}_g(e_1, e_2)$), where e_1 is the expression finding groups, and e_2 is the main expression to aggregate over, conditioned on the groups found by evaluating e_1 ,
- (4) Sampling ($\psi_\mu(e)$) where μ denotes a sampler,
- (5) Group-by sampling ($\hat{\psi}_\mu(e_1, e_2)$), which is sampling but similar to aggregate, we have e_1 being the expression finding groups.

Remark. The One-overwrite $\mathbb{1}(e)$ is particularly useful for defining magic-set transformation under tagged semantics. Overwriting **1** will make the tuples in magic-set predicate serve as pure demand fact for optimization, not conditioned on some given tags.

B.3 Operational Semantics of SCLRAM

We show the full semantics in Fig. 23 and Fig. 24, with the addition of group-by, sampling, and early-removal. Note that the sampler $\mu : \mathcal{U}_{\mathbb{R}} \rightarrow \mathcal{U}$ takes in facts tagged by weights, and samples according to the weight. The early removal happens in the stage of normalization before each rule update.

B.4 Provenance Structures

We provide detailed runtime analysis for each presented provenance structure in the following sections. Additionally, we discuss the extended provenance diff-top-k-proofs-me which can handle mutual exclusive facts.

B.4.1 diff-max-min-prob.

Runtime Analysis. We showcase the time complexity for each operator in the max-min-prob provenance structure in Table 6. We next present our optimized counting algorithm in Fig. 1 which performs counting over a set of max-min-prob tagged-tuples. Note that we only showed the algorithm for mmp for simplicity. But it easily extends to dmmp. It can be easily shown that its runtime complexity is $O(n \log(n))$.

B.4.2 diff-add-mult-prob.

Runtime Analysis. We showcase the time complexity for the diff-add-mult-prob provenance structure in Table 7. We denote the number of total variables in the input as n . Since we are performing calculations over the dual-numbers, the time complexity is decided by the gradient calculation over vectors of dimension n .

Expression semantics

$\alpha : \mathbb{U} \rightarrow \mathbb{U}, \quad \beta : \mathbb{U} \rightarrow \text{Bool}, \quad g : \mathcal{U} \rightarrow \mathcal{U}, \quad \mu : \mathcal{U}_{\mathbb{R}} \rightarrow \mathcal{U}, \quad \llbracket e \rrbracket : \mathcal{F}_T \rightarrow \mathcal{U}_T$
--

$$\begin{array}{c}
 \frac{}{\llbracket \emptyset \rrbracket(F_T) = \emptyset} \text{(EMPTY SET)} \quad \frac{t :: p(u) \in F_T}{t :: u \in \llbracket p \rrbracket(F_T)} \text{(PREDICATE)} \\
 \\
 \frac{t :: u \in \llbracket e \rrbracket(F_T)}{0 :: u \in \llbracket 0(e) \rrbracket(F_T)} \text{(0-OVERWRITE)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T)}{1 :: u \in \llbracket 1(e) \rrbracket(F_T)} \text{(1-OVERWRITE)} \\
 \\
 \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad \beta(u) = \text{true}}{t :: u \in \llbracket \sigma_{\beta}(e) \rrbracket(F_T)} \text{(SELECT)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad u' = \alpha(u)}{t :: u' \in \llbracket \pi_{\alpha}(e) \rrbracket(F_T)} \text{(PROJECT)} \\
 \\
 \frac{t :: u \in \llbracket e_1 \rrbracket(F_T) \cup \llbracket e_2 \rrbracket(F_T)}{t :: u \in \llbracket e_1 \cup e_2 \rrbracket(F_T)} \text{(UNION)} \quad \frac{t_1 :: u_1 \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u_2 \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes t_2) :: (u_1, u_2) \in \llbracket e_1 \times e_2 \rrbracket(F_T)} \text{(PRODUCT)} \\
 \\
 \frac{t_1 :: u \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes t_2) :: u \in \llbracket e_1 \cap e_2 \rrbracket(F_T)} \text{(INTERSECT)} \\
 \\
 \frac{t_1 :: (u, u_1) \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: (u, u_2) \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes t_2) :: (u, u_1, u_2) \in \llbracket e_1 \bowtie e_2 \rrbracket(F_T)} \text{(NATURAL-JOIN)} \\
 \\
 \frac{t :: u \in \llbracket e_1 \rrbracket(F_T) \quad \llbracket e_2 \rrbracket(F_T) \not\models u}{t :: u \in \llbracket e_1 - e_2 \rrbracket(F_T)} \text{(DIFF-1)} \quad \frac{t_1 :: u \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes (\ominus t_2)) :: u \in \llbracket e_1 - e_2 \rrbracket(F_T)} \text{(DIFF-2)} \\
 \\
 \frac{t :: (u_1, u_2) \in \llbracket e_1 \rrbracket(F_T) \quad \llbracket e_2 \rrbracket(F_T) \not\models u_1}{t :: (u_1, u_2) \in \llbracket e_1 \triangleright e_2 \rrbracket(F_T)} \text{(ANTIJOIN-1)} \\
 \\
 \frac{t_1 :: (u_1, u_2) \in \llbracket e_1 \rrbracket(F_T) \quad t_2 :: u_1 \in \llbracket e_2 \rrbracket(F_T)}{(t_1 \otimes (\ominus t_2)) :: (u_1, u_2) \in \llbracket e_1 \triangleright e_2 \rrbracket(F_T)} \text{(ANTIJOIN-2)} \\
 \\
 \frac{X_T \subseteq \llbracket e \rrbracket(F_T) \quad \{t_i :: u_i\}_{i=1}^n = X_T \quad \{\bar{t}_j :: \bar{u}_j\}_{j=1}^m = \llbracket e \rrbracket(F_T) - X_T \quad u \in g(\{u_i\}_{i=1}^n)}{(\bigotimes_{i=1}^n t_i) \otimes (\bigotimes_{j=1}^m (\ominus \bar{t}_j)) :: u \in \llbracket \gamma_g(e) \rrbracket(F_T)} \text{(AGGREGATE)} \\
 \\
 \frac{\begin{array}{l} t' :: (u_g, u'_g) \in \llbracket e_1 \rrbracket(F_T) \quad X_T \subseteq \llbracket e_2 \rrbracket(F_T) \\ \{t_i :: (u_g, u_i)\}_{i=1}^n = X_T \quad \{\bar{t}_j :: (u_g, \bar{u}_j)\}_{j=1}^m = \llbracket e_2 \rrbracket(F_T) - X_T \quad u \in g(\{u_i\}_{i=1}^n) \end{array}}{t' \otimes (\bigotimes_{i=1}^n t_i) \otimes (\bigotimes_{j=1}^m (\ominus \bar{t}_j)) :: (u_g, u'_g, u) \in \llbracket \gamma_g(e_1, e_2) \rrbracket(F_T)} \text{(GROUP-BY AGGREGATE)} \\
 \\
 \frac{\{t_i :: u_i\}_{i=1}^n = \llbracket e \rrbracket(F_T) \quad u_j \in \mu(\{\text{weight}(t_i) :: u_i\}_{i=1}^n)}{t_j :: u_j \in \llbracket \psi_{\mu}(e) \rrbracket(F_T)} \text{(SAMPLE)} \\
 \\
 \frac{t' :: (u_g, u'_g) \in \llbracket e_1 \rrbracket(F_T) \quad \{t_i :: (u_g, u_i)\}_{i=1}^n \subseteq \llbracket e \rrbracket(F_T) \quad u_j \in \mu(\{\text{weight}(t_i) :: u_i\}_{i=1}^n)}{(t' \otimes t_j) :: (u_g, u'_g, u_j) \in \llbracket \hat{\psi}_{\mu}(e_1, e_2) \rrbracket(F_T)} \text{(GROUP-BY SAMPLE)}
 \end{array}$$

Fig. 23. Operational semantics of SCLRAM.

Algorithm 1: Counting over max-min-prob tagged tuples

Data: $U_{\text{mmp}} = \{t_1 :: u_1, t_2 :: u_2, \dots, t_n :: u_n\} : \mathcal{U}_{\text{mmp}}$, set of tagged-tuples
Result: $U'_{\text{mmp}} : \mathcal{U}_{\text{mmp}}$

```

/* sort all positive tuples according to their tags from small to large. O(nlog(n)) */
1  $\mathbf{t}^{\text{pos}} = \text{sorted}([t_i \mid i = 1 \dots n]);$ 
2  $\mathbf{t}^{\text{neg}} = [1 - t_{n-i+1}^{\text{pos}} \mid i = 1 \dots n];$ 
/* Iterate through all possible partitions between positive and negative tags. O(n) */
3  $U'_{\text{mmp}} = \{t_n^{\text{neg}} :: 0, t_1^{\text{pos}} :: n\};$ 
4 for  $i = 1 \dots (n - 1)$  do
5    $\lfloor$  Add  $\min(t_{i+1}^{\text{pos}}, t_i^{\text{neg}}) :: (n - i)$  to  $U'_{\text{mmp}};$ 
6 return  $U'_{\text{mmp}}$ 
    
```
